FIX Protocol Mastery Guide (QuickFIX, C++)

Study guide for a C++ developer learning FIX with QuickFIX on Windows, then Linux. Built from the first practice sessions, November 2026.

1. Why FIX exists

A trading firm and a broker run different systems written by different companies. They must agree on how to say buy 100 AAPL at 150.50, how to report a fill or a reject, and what to do when the network drops. Before FIX every broker had a private format. FIX is the shared standard, like SMTP for email.

FIX has two layers:

1. **Session layer**: are we connected, did I miss a message. Logon, Heartbeat, sequence numbers, resend.
2. **Application layer**: the business content. Orders, fills, cancels, market data.

The standard is maintained by the FIX Trading Community (fixtrading.org). FIX is the protocol, QuickFIX is a library that implements it, and simulators such as FIXSIM are separate third-party tools.

2. The trading chain

Trader or algo -> OMS -> FIX -> broker -> exchange. Results flow back the same way as ExecutionReports.

- The trader sends new orders, cancels and changes. The broker or exchange sends back acks, fills and rejects.
- The OMS (order management system) stores every order and its state, and sends and receives the FIX messages.
- There can be two FIX sessions in a chain: trader to broker and broker to exchange. Each has its own sequence numbers.
- In your demo the initiator plays the OMS or client and the acceptor plays the broker or exchange.

Things that are not FIX:

- **Zerodha and Indian retail brokers**: Kite Connect is REST plus WebSocket. As far as I know NSE and BSE use their own binary interfaces. Check current documents before relying on this.
- **Dow Jones** is an index, not an exchange. Exchanges are NYSE and Nasdaq.
- **MPC custody (cb-mpc)** decides who may sign and move crypto. FIX only handles the trading step. Deposits and withdrawals are on-chain or REST.

Where FIX is used: funds and prop firms to brokers, futures exchanges such as CME, many crypto exchanges for institutional clients, and inside firms between OMS, algos and execution.

3. Wire format

A message is tag=value pairs separated by the invisible SOH character (0x01). Example, with SOH shown as a pipe:

```
8=FIX.4.4|9=121|35=D|34=2|49=CLIENT|52=20261106-
11:50:33.330|56=EXEC|11=ORD1|38=100|40=2|44=150.5|54=1|55=AAPL|59=0|60=202611
06-11:50:33|10=207|
```

QuickFIX writes header fields first (8, 9, 35, 34, 49, 52, 56), then body fields sorted by tag, then the trailer (10). Body length (9) and checksum (10) are calculated for you. The checksum is the sum of all bytes before 10= modulo 256, as three digits.

Core tags to know by heart

Group	Tags
Header	8 BeginString, 9 BodyLength, 35 MsgType, 34 MsgSeqNum, 49 SenderCompID, 56 TargetCompID, 52 SendingTime, 43 PossDupFlag, 122 OrigSendingTime
Trailer	10 CheckSum
Logon	98 EncryptMethod, 108 HeartBtInt, 553 Username, 554 Password
Order	11 ClOrdID, 41 OrigClOrdID, 55 Symbol, 54 Side, 38 OrderQty, 40 OrdType, 44 Price, 59 TimeInForce, 60 TransactTime

Group	Tags
Execution	37 OrderID, 17 ExecID, 39 OrdStatus, 150 ExecType, 14 CumQty, 151 LeavesQty, 32 LastQty, 31 LastPx, 6 AvgPx
Resend	7 BeginSeqNo, 16 EndSeqNo, 36 NewSeqNo, 123 GapFillFlag
Rejects	58 Text, 45 RefSeqNum, 372 RefMsgType, 380 BusinessRejectReason

Core values

Field	Values
54 Side	1 Buy, 2 Sell
40 OrdType	1 Market, 2 Limit, 3 Stop, 4 StopLimit
59 TimeInForce	0 Day, 1 GTC, 3 IOC, 4 FOK
39 OrdStatus	0 New, 1 PartiallyFilled, 2 Filled, 4 Canceled, 5 Replaced, 6 PendingCancel, 8 Rejected, A PendingNew, C Expired, E PendingReplace
150 ExecType (FIX 4.4)	0 New, 4 Canceled, 5 Replaced, 8 Rejected, C Expired, F Trade, G TradeCorrect, H TradeCancel

Tag 39 is the state of the order. Tag 150 is what just happened. ClOrdID (11) is the client's id. OrderID (37) is the exchange's id. Real systems map between them constantly.

Message types (tag 35)

Type	Name	Layer
A	Logon	Session
0	Heartbeat	Session
1	TestRequest	Session
2	ResendRequest	Session

Type	Name	Layer
4	SequenceReset (GapFill)	Session
3	Reject	Session
5	Logout	Session
D	NewOrderSingle	Application
F	OrderCancelRequest	Application
G	OrderCancelReplaceRequest	Application
H	OrderStatusRequest	Application
q	OrderMassCancelRequest	Application
8	ExecutionReport	Application
9	OrderCancelReject	Application
j	BusinessMessageReject	Application
V, W, X	Market data request, snapshot, incremental	Application

Nobody memorizes the whole specification. Learn about 30 codes by repetition, and look up the rest in FIXimate on fixtrading.org or in the exchange's own spec.

4. The session layer in depth

1. The initiator opens a TCP connection and sends Logon (35=A) with its HeartBtInt (108).
2. The acceptor checks the CompIDs and replies with its own Logon. Both sides now fire onLogon.
3. When a side has been quiet for the heartbeat interval it sends a Heartbeat (35=0). If a side hears nothing it sends a TestRequest (35=1). No answer means the connection is dead.
4. Every message carries a sequence number (34). Each direction counts separately, and heartbeats count too.
5. **Too high**: a gap. The receiver sends ResendRequest (35=2) with 7=BeginSeqNo and 16=EndSeqNo, where 0 means to infinity. Messages that arrive meanwhile are queued.

6. **Session messages are never resent.** They are replaced by SequenceReset-GapFill (35=4, 123=Y) with 36=NewSeqNo, meaning skip ahead to this number.
7. **Business messages are resent** with 43=Y (PossDupFlag) and 122=OrigSendingTime. This is why a real OMS needs a check in toApp: after a long outage you may not want to replay a stale order.
8. **Too low** without PossDup is serious: the session is rejected and disconnected.
9. The store (sequence numbers plus sent messages) is what makes recovery possible. Deleting only one side's store is the classic way to create a mismatch.

In your own logs you saw this: MsgSeqNum too high, ResendRequest FROM 15 TO 0, SequenceReset to 17, then Processing QUEUED message 17.

5. The order lifecycle (heart of an OMS)

Happy path: NewOrderSingle -> ExecutionReport New (150=0, 39=0) -> partial fills (150=F, 39=1) -> final fill (150=F, 39=2).

Other paths:

- **Reject**: ExecutionReport with 150=8, 39=8 and a reason in 58.
- **Cancel**: OrderCancelRequest (35=F) with 41=OrigClOrdID and a new 11=ClOrdID. Success returns ExecutionReport 150=4. Failure (already filled, unknown order) returns OrderCancelReject (35=9).
- **Replace**: OrderCancelReplaceRequest (35=G). It is not an edit. If accepted, the old ClOrdID is replaced by the new one. Chaining of 41 and 11 is a common source of bugs.
- **Races**: a fill can arrive before the ack, a cancel can cross a fill, duplicate ExecIDs can appear, and an order can expire at end of day.

States: PendingNew, New, PartiallyFilled, Filled, PendingCancel, Canceled, PendingReplace, Replaced, Rejected, Expired. Illegal transitions must be detected and logged, not silently accepted.

The three questions for every concept: what real situation causes it, which message carries it, and what can go wrong.

6. QuickFIX architecture

Piece	Job
FIX::Application	Interface with six callbacks: onCreate, onLogon, onLogout, toAdmin, fromAdmin, toApp, fromApp
FIX::MessageCracker	crack() reads tag 35 and calls the matching onMessage overload
SessionSettings	Reads the .cfg file
FileStoreFactory	Keeps sequence numbers and sent messages (store folder)
FileLogFactory	Writes raw messages and events (log folder)
SocketInitiator / SocketAcceptor	Run TCP and session logic on their own threads
FIX::Session::sendToTarget	Sends a message: adds header, length, checksum, stores it, writes it to the socket
FIX44.xml data dictionary	Defines which fields each message may or must have, so QuickFIX can validate. A bad message gets a Reject

Notes that cost time to find:

- In QuickFIX 1.16 the engine constructor order is (application, storeFactory, settings, logFactory).
- A message type you did not override makes crack() throw UnsupportedMessageType, and QuickFIX replies with BusinessMessageReject (35=j, 380=3).
- Always handle reject message types on the receiving side, or the two sides can answer each other with rejects.
- admin means session plumbing (logon, heartbeat, resend). app means business messages (orders, fills).
- Session identity is BeginString plus SenderCompID plus TargetCompID. Configs are mirror images: my Sender is your Target.

Threads (important for the job)

- Callbacks run on QuickFIX threads, not on main.

- Never block inside a callback. A slow database call stalls heartbeats.
- Shared state such as an order map needs a mutex or a handoff queue.
- Do not hold your own mutex while calling sendToTarget. A QuickFIX thread may hold the session lock and wait for your mutex while you wait for the session lock. That is a deadlock.
- Production designs push the message onto a queue in fromApp and process it on a worker thread.

7. Your project, step by step

Files: CMakeLists.txt, acceptor.cfg, initiator.cfg, acceptor.cpp, initiator.cpp, spec/FIX44.xml. Source on GitHub: marvic41/fix-quickfix-oms.

Install and build on Windows

1. Install Visual Studio 2022 Build Tools with the workload Desktop development with C++, and Git.
2. Install vcpkg and QuickFIX:

```
git clone https://github.com/microsoft/vcpkg C:/vcpkg
C:/vcpkg/bootstrap-vcpkg.bat
C:/vcpkg/vcpkg install quickfix:x64-windows
```

3. Create the project folder, make a spec folder, and download FIX44.xml with curl.exe (plain curl in PowerShell 5 is an alias of Invoke-WebRequest).
4. Build from the project folder:

```
cmake -B build -S . -
DCMAKE_TOOLCHAIN_FILE=C:/vcpkg/scripts/buildsystems/vcpkg.cmake
cmake --build build --config Release
```

5. Run both programs from the project folder, because config paths are relative. Start the acceptor first, then the initiator.

No IDE is required. Visual Studio can open the generated .sln in the build folder for debugging. Set the Working Directory of each project to the project folder, and use the Debug configuration.

Problems met and their fixes

Problem	Cause and fix
curl cannot find path	The spec folder did not exist. Create it first
cmake not recognized	vcpkg unzips its own tools and does not add them to PATH. Install cmake with winget or add its bin folder to PATH
Constructor error C2665	Argument order. Use (app, store, settings, log)
Cannot run build/Release/initiator.exe	Wrong folder. Run from the project root with ./build/Release/...
Permission denied on store_client	Another running process holds the file. Check which exe is running, and kill leftovers
Two fills for one order	A stale store replayed an old order after a gap. Delete both store folders for a clean start, and add a duplicate ClOrdID check
OrderID and ExecID differ	The counter was incremented twice in one expression. Increment once into a variable
Old behaviour after a rebuild	A different configuration was built, or an old process kept running. Do a clean build and stop old processes

Known simulator limits (to fix in Week 2 and 3)

- The acceptor fills everything completely and instantly.
- A market order fills at price 0, because the simulator has no price source.
- OrderID and ExecID restart at 1 on every run. A real exchange never reuses them.
- No duplicate ClOrdID check. No cancel or replace handling.

8. Reading the logs

Two kinds of files per session. The event log is a diary (logon, resend, logout). The messages log holds the raw FIX messages of both directions, so tag 49 tells you who sent each line.

Make a log readable by replacing SOH with a pipe:

```
(Get-Content log_client/*messages*.log) -replace [char]1,'|'
```

Useful searches:

```
(Get-Content log_exec/*messages*.log) -replace [char]1,'|' | Select-String
'35=0'
(Get-Content log_exec/*messages*.log) -replace [char]1,'|' | Select-String
'35=D'
(Get-Content log_exec/*messages*.log) -replace [char]1,'|' | Select-String
'43=Y'
(Get-Content log_exec/*messages*.log) -replace [char]1,'|' | Select-String
'49=EXEC'
```

How to read what you find:

- An order and its fill are two messages linked by tag 11. They never appear in one line.
- Sequence numbers of the two sides are separate counters. Equal numbers on two lines are a coincidence.
- A gap in one sender's 34 values usually means heartbeats. Check with the 35=0 search.
- In a reject, 45 is the sequence number of your message that was rejected, and 372 is its type.
- Heartbeats only appear when a side is idle for HeartBtInt. For a quick test set HeartBtInt=5.

9. What is still missing in the demo

Area	Missing pieces
Actions	Replace (35=G), order status request (35=H), mass cancel (35=q)
Order types	Stop and StopLimit
Time in force	GTC, IOC, FOK. IOC and FOK are quick wins because they produce partial fills and cancels
Execution reports	New ack, partial fills, reject, canceled, replaced, expired, trade correct or bust

Area	Missing pieces
Other FIX areas	Market data (V, W, X), positions and account queries, allocations and confirmations

10. From demo to production

Demo	Production
Orders typed in a console	Orders from strategies, GUI, API or a queue
Orders in a std::map	Persistent order store, reloaded on restart
Work inside the callback	fromApp enqueues, worker threads process
Basic status tracking	Strict state machine, duplicate and out-of-order protection
File store and file log	Faster or custom stores, asynchronous logging
One session	Many sessions, one per venue, each with its own spec
Localhost TCP	TLS, VPN or leased lines, firewalls
Password in a config file	Secrets management
Console prints	Metrics, alerts, dashboards
Manual testing	Unit tests, session replay, venue certification tests
Run from PowerShell	Linux services, daily session reset, clock sync

A job like improve and extend a production OMS means reading a large codebase, finding where order state is updated, and fixing edge cases and performance. Your own project is the practice ground for that.

11. Languages and engines

Language	Typical use in trading
C++	Low latency parts: gateways, matching engines, HFT, order routers
Java	Banks, brokers, many OMS and EMS platforms. QuickFIX/J is the Java version
C# and .NET	Some banks and vendors
Python	Research, test tools, simulators, glue code. Rarely the hot path
Go and Rust	Growing, especially at crypto firms
kdb+ and q	Market data storage and analytics

QuickFIX versus commercial engines such as OnixS or Cameron. QuickFIX is free and open source, data dictionary driven, and good enough for most connectivity work. It is not the lowest latency option. Commercial engines mainly sell lower latency, vendor support and ready made venue certifications. Whether one is ten times better is a claim I cannot verify, and it depends on the workload. For learning the concepts, QuickFIX is the right choice, because session, orders and ExecutionReports are the same in every engine. At the very lowest latencies firms avoid FIX on the hot path and use binary exchange protocols and custom parsers. On any real job, ask which engine and version the system already uses.

12. Two month plan

Weeks 1 to 3 (agreed plan)

- Week 1: debugger and logs, interactive console, logout and reconnect, credentials in Logon (Username and Password in toAdmin and fromAdmin), session schedule, own event log, clean settings, commit daily.
- Week 2: order store and state machine, ExecutionReport handling by 150 and 39, cancel and replace, all reject types, duplicate and out-of-order protection. Acceptor: ack, partial fills, configurable behaviour, cancel and replace logic including too late to cancel.

- Week 3: sequence recovery tests, toApp guard for PossDup, queue between fromApp and a worker, latency measurement (p50 and p99), WSL2 Ubuntu build with gdb and perf, unit tests, README with log excerpts.

Weeks 4 to 8 (deepening)

- Week 4: Linux as the main environment. CMake, gdb, perf, valgrind or sanitizers. Run the same project under systemd.
- Week 5: performance. Allocation hotspots, logging cost, store choice, lock contention, histograms of latency. Compare file store and memory store.
- Week 6: a real counterparty. Use a crypto exchange test environment or a FIX simulator, read its spec PDF, and implement its custom tags and quirks. Add a market data session.
- Week 7: FIX 5.0 SP2 with FIXT 1.1, a custom data dictionary with custom tags, FIX Orchestra rules of engagement.
- Week 8: polish and apply. Read an open source OMS or FIX codebase. Prepare proposals with the repo link, and keep applying to small integration gigs.

Apply rule: apply once partial fills and rejects, cancel and replace, restart recovery, a Linux build and a README with log excerpts all work in the repo, whether or not you feel ready.

Daily routine: first hour read or look up a concept, then build one feature, then 20 minutes of notes and a git commit.

13. Questions you should be able to answer

Question	Short answer
What is FIX and why is it used	A shared messaging standard for trading, so systems from different firms connect with configuration instead of custom work
Session layer versus application layer	Session keeps the connection reliable (logon, heartbeat, sequence, resend). Application carries orders, fills and market data
What is a sequence number for	Detects lost or duplicated messages. A gap triggers a resend request

Question	Short answer
What happens when a sequence number is too high	Queue newer messages, send ResendRequest, then GapFill or resend with PossDup
What happens when it is too low	Serious. Without PossDup the session is rejected and disconnected
Why are admin messages not resent	They are only meaningful at the moment. They are replaced by a GapFill
What does PossDupFlag mean	The message is a replay. The receiver must be able to ignore duplicates, and the sender may choose not to replay stale orders
ClOrdID versus OrderID	ClOrdID is the client's id, OrderID is the exchange's id. Fills echo ClOrdID so the OMS can match them
OrdStatus versus ExecType	OrdStatus is the order's state now. ExecType is the event that produced the report
How does cancel replace work	The new request has its own ClOrdID and points to the old one with OrigClOrdID. The ids chain on success
What if a cancel and a fill cross	The venue answers with OrderCancelReject, and the OMS must keep state consistent
How do you prevent a duplicate fill from being processed twice	Track ExecIDs per order and ignore repeats
How do you refuse a duplicate order	The acceptor checks ClOrdID before processing. The OMS never reuses ids, for example by using a start time prefix
Why do callbacks need care	They run on QuickFIX threads. Blocking delays heartbeats, shared data needs a mutex, and holding your own lock while sending can deadlock
What goes in toAdmin and fromAdmin	Credentials on Logon, and validation of the counterparty

Question	Short answer
What goes in toApp	A last chance to veto, for example stale orders being replayed
What does the store do	Persists sequence numbers and sent messages so a restart resumes and resends correctly
What is the data dictionary for	Validates messages against the spec. Each venue has its own variant with custom tags
How would you measure latency	Timestamp at the socket read, in fromApp, and at the send. Report min, p50, p99 over many orders
How would you speed up a QuickFIX system	Queue between fromApp and workers, avoid allocations and slow logging in hot paths, choose a faster store, pin threads, profile with perf
Why is FIX not used for everything	At the lowest latency firms use binary exchange protocols. Retail brokers often offer REST and WebSocket
What would you ask a client first	Which exchanges and FIX versions, which engine, and the most painful order handling or latency problem

14. Honest positioning for proposals

Say what is true: strong C++ on Linux, profiling and porting work, multi-coin integration experience, MPC custody, and a QuickFIX project with the full order lifecycle. Do not claim production OMS experience until you have it. Offer a small paid task so the client can judge your work. Keep proposals short and specific, and link the GitHub repo once its README exists.

15. References

- fixtrading.org: specifications, FIXimate lookup, session layer profiles, FIX Orchestra
- QuickFIX documentation and the examples folder (executor, ordermatch, tradeclient)

- Each exchange or broker's own FIX spec PDF, which is where real integration work happens

16. Notes template

For every new concept write three lines: the situation that causes it, the message that carries it, and what can go wrong. These lines become your README sections and your interview answers.